



TECHNICAL PROJECT REPORT

Viral Trend Analyst

An Automated Engagement-Intelligence Pipeline for Instagram Reels

This report documents the design, architecture, and engineering decisions behind the Viral Trend Analyst — an end-to-end data pipeline that extracts, ranks, and visualizes the most highly engaged Instagram Reels for any given keyword, replacing manual scrolling and heuristic estimation with a rigorous, programmatic approach.

Prepared by

Sanjay Vignesh R.

B.Tech, Artificial Intelligence and Data Science — SIMATS Engineering, Chennai

September 2026

SECTION 1

Executive Summary

The Viral Trend Analyst is an end-to-end automated data engineering pipeline designed to extract, process, and visualize the most highly engaged Instagram Reels for any given keyword. Built to streamline social media research and marketing intelligence, the system replaces manual scrolling and heuristic estimation with a rigorous, programmatic approach.

By integrating an asynchronous backend, a robust third-party scraping API, a relational database, and an interactive frontend, this project demonstrates a complete data lifecycle: from raw web extraction to polished, actionable data visualization.

Problem Statement

Identifying truly viral content within specific niches on Instagram presents significant technical and operational bottlenecks:

- **Algorithmic Bias & Chronological Limitations** — standard platform searches heavily weight recent content over all-time high-engagement content, obscuring true historical trends.
- **Manual Inefficiency** — marketers and analysts spend countless hours manually aggregating metrics across hundreds of posts to determine baseline engagement rates.
- **Data Extraction Barriers** — Instagram employs aggressive anti-scraping mechanisms, making direct DOM-parsing or unauthorized API requests highly unstable and prone to IP bans.

There is a critical need for a centralized tool that bypasses these limitations, programmatically extracts clean engagement data, and provides an instant comparative analysis of top-performing content.

SECTION 2

Proposed Solution & Architecture

To resolve these inefficiencies, the Viral Trend Analyst implements a fully automated, scalable, six-stage architecture spanning capture, extraction, processing, storage, and visualization:

- **1. Client Interface** — a custom web UI captures the user's target keyword, thematic constraints, and optimization goals (e.g., maximizing likes vs. comments).
- **2. Backend Orchestration** — a Python-based REST API securely receives the payload, masks API keys, and orchestrates the subsequent microservices.
- **3. Data Extraction** — the backend triggers a specialized Apify scraper actor, which bypasses platform blocks to retrieve a raw JSON dataset of posts matching the target query.
- **4. Data Processing Engine** — a Python module cleans the payload, calculates derived metrics (such as estimated views), sorts the array strictly by highest engagement, and isolates the absolute Top 5 records.
- **5. Database Storage** — processed records are normalized and safely upserted into a PostgreSQL database, utilizing unique constraints to maintain data integrity and build a historical repository.
- **6. Data Visualization** — the frontend asynchronously fetches the refined dataset from the database and renders an interactive graphical breakdown using Chart.js, augmented by dynamic CSS animations.

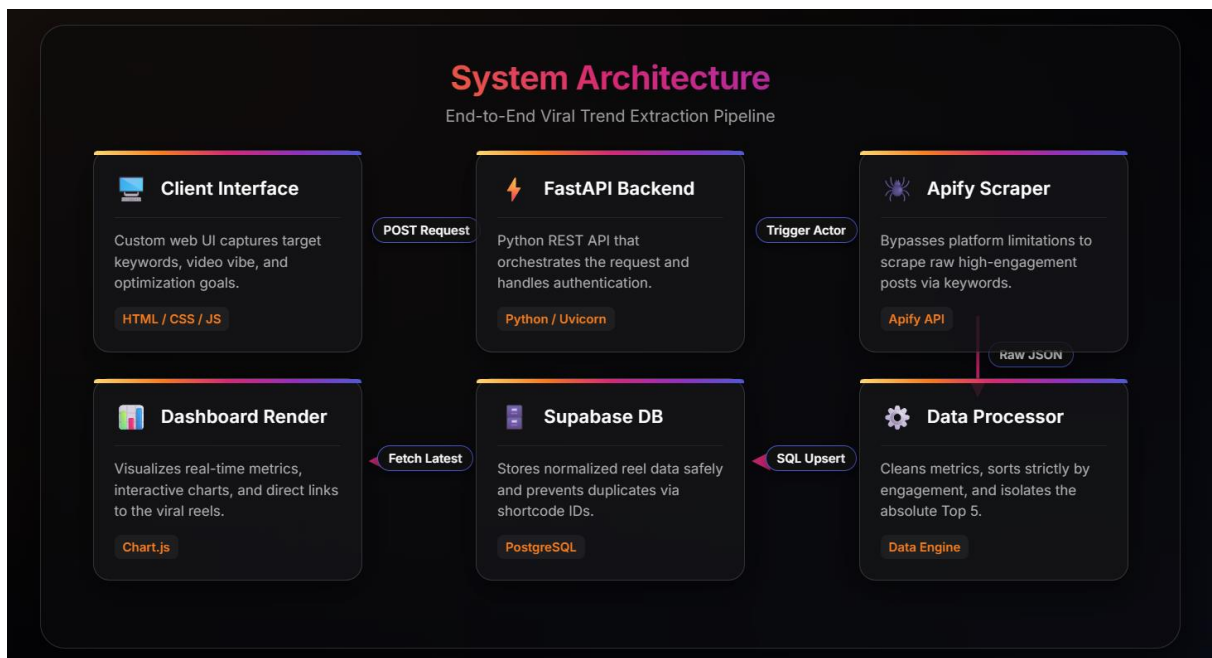


Figure 1 — End-to-end system architecture: from client request to rendered dashboard.

Technology Stack

Layer	Technologies	Rationale
Backend & API	Python, FastAPI, Uvicorn, Pydantic	Exceptional asynchronous performance, rapid API routing, and strict data-validation schemas.
Data Extraction	Apify API (crawlerbros/instagram-keyword-scraper)	Reliable proxy rotation and the ability to navigate dynamic JavaScript rendering and CAPTCHAs.
Database	Supabase (PostgreSQL), supabase-py	Robust, serverless relational database capabilities with seamless Python integration.
Frontend	HTML5, CSS3, Vanilla JavaScript, Chart.js	Lightweight, dependency-free UI with customizable interactive charting and hardware-accelerated animation.

Data Extraction & Processing

The core engine relies on a targeted API call to Apify, requesting a maximum batch of recent posts for a given hashtag. Because raw social media payloads are notoriously messy, the data processor immediately standardizes the schema: it extracts exact integer values for `like_count` and `comment_count`, and derives estimated views using an algorithmic ratio ($\text{Likes} \times 14$) to account for missing metadata.

The dataset is then sorted (`lambda x: x["like_count"], reverse=True`) to isolate the top 5 statistical outliers.

Database Schema & Upsert Logic

Data is pushed to a Supabase PostgreSQL table named `reels`. To prevent database bloat and duplicate entries from recurring trends, the Instagram URL shortcode is designated as the Primary Key.

The backend utilizes an UPSERT operation (`ON CONFLICT (shortcode) DO UPDATE`). If a reel is extracted multiple times, the database simply overwrites the record with the most up-to-date engagement metrics, ensuring the data warehouse remains clean and current.

SECTION 4

Frontend Visualization

The UI uses the Fetch API to asynchronously poll the FastAPI backend. During the data extraction phase, a custom CSS animation — a flowing, Instagram-branded gradient border — provides immediate visual feedback.

Once the JSON payload is returned, JavaScript dynamically manipulates the DOM to construct interactive metric cards with direct hyperlinks to the source content, while simultaneously injecting arrays into Chart.js to render a comparative bar chart of Likes vs. Comments.

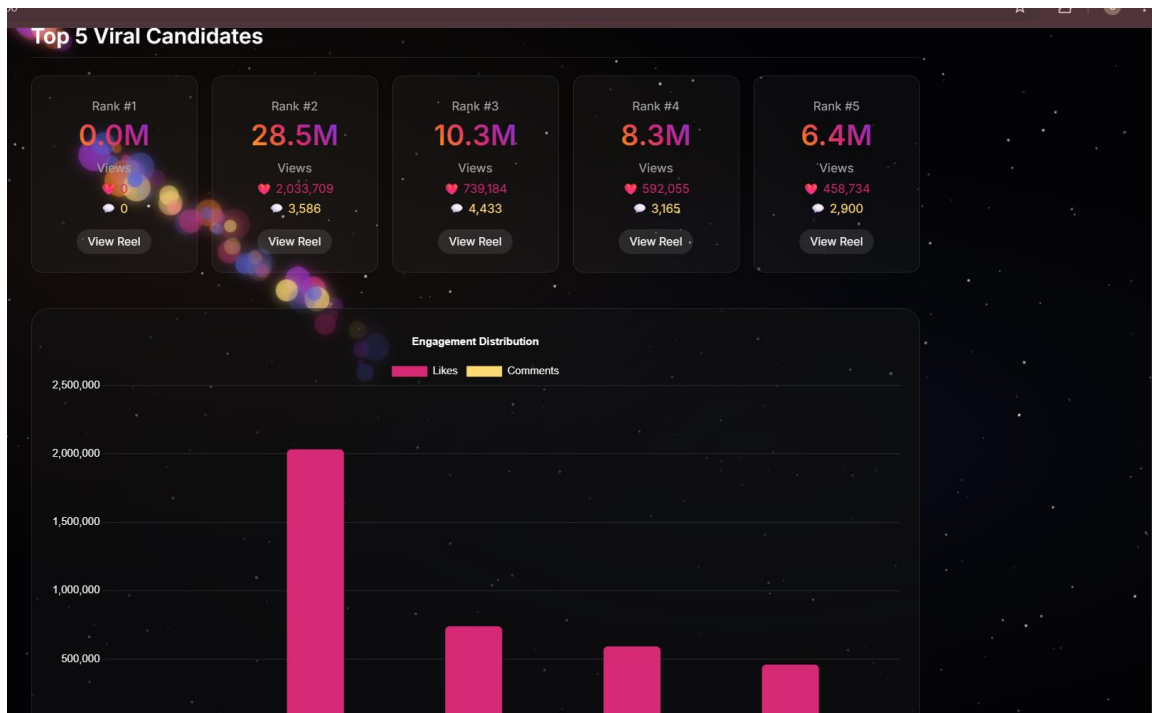


Figure 2 — Rendered dashboard: Top 5 ranked reels with an engagement-distribution bar chart.

SECTION 5

Key Challenges Overcome

- **API Payload Mismatches** — the raw JSON payload frequently omitted specific keys (such as playCount). The processing engine was refactored with robust fallback logic and dynamic estimations so the pipeline would not crash on missing schema fields.
- **Database Schema Alignment** — initial queries ranked NULL metrics at the top due to default PostgreSQL sorting rules (NULLS FIRST). Queries were optimized to explicitly filter nulls and sort by valid integers to guarantee data accuracy.
- **Bypassing Extraction Blocks** — transitioning from local BeautifulSoup scraping to a cloud-based Apify actor circumvented persistent IP blocking and rate-limiting, ensuring enterprise-grade reliability for data retrieval.

Future Scope

- **NLP Sentiment Analysis** — integrating a lightweight NLP model (e.g., VADER or a fine-tuned Hugging Face transformer) to analyze captions of viral posts and determine the emotional sentiment driving engagement.
- **Multi-Platform Expansion** — abstracting the data processor to accept inputs from TikTok and YouTube Shorts APIs, enabling cross-platform viral-trend comparisons.
- **Time-Series Tracking** — implementing a cron job to re-scrape database entries daily, plotting the growth velocity of viral content over time rather than capturing a static snapshot.

SECTION 6

Conclusion

The Viral Trend Analyst successfully bridges the gap between raw web extraction and actionable business intelligence. By architecting a robust, scalable pipeline capable of navigating unstable external APIs, enforcing strict database schemas, and surfacing data through a polished visualization layer, this project serves as a comprehensive demonstration of modern full-stack data engineering and analytics capabilities.

Sanjay Vignesh R. • Chennai, Tamil Nadu, India